

Outline

The two failure modes

The decomposition

Levers and the tradeoff

Diagnosis

Fixing each problem

Modern wrinkle: double descent

Wrap-up

Underfit vs. overfit

Models fail in two opposite ways:

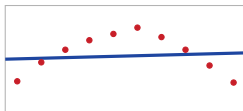
Underfitting. Model class too rigid to capture the signal.

- ▶ High training error AND high test error.
- ▶ Adding data won't help.
- ▶ Fix: more flexible model, more features, less regularization.

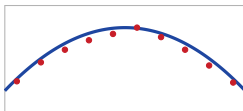
Overfitting. Model so flexible it memorizes noise.

- ▶ Low training error, high test error.
- ▶ More data WILL help.
- ▶ Fix: simpler model, fewer features, more regularization, more data.

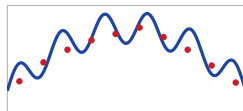
underfit (degree 1)



good fit (degree 3)



overfit (degree 9)



Predict-first: where does the error come from?

Same data, three fits. The training error tells us nothing useful by itself.

Predict-first: where does the error come from?

Same data, three fits. The training error tells us nothing useful by itself.

Predict: for each fit above, decompose the error into “the model class can’t represent the truth” vs. “the model overreacted to noise in this specific sample”.

Predict-first: where does the error come from?

Same data, three fits. The training error tells us nothing useful by itself.

Predict: for each fit above, decompose the error into “the model class can’t represent the truth” vs. “the model overreacted to noise in this specific sample”.

- ▶ **Degree 1** (underfit): error is mostly from *model class*. A straight line cannot bend. Different training data $\rightarrow$ similar wrong fit. Low variance, high systematic error.
- ▶ **Degree 9** (overfit): error is mostly from *specific training sample*. The wiggles chase noise. Different training data $\rightarrow$ wildly different fits. High variance, low systematic error.
- ▶ **Degree 3** (sweet spot): small contribution from each. The model is flexible enough to capture the truth without chasing noise.

Systematic error = **bias**. *Sensitivity to the specific training sample* = **variance**.

Bias and variance: definitions

Setup: true relationship $y = f(\mathbf{x}) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$. A training set D is randomly drawn; you train a model $\hat{f}_D$ on it.

Imagine repeating training many times on independent D 's. Define the **average model** at a fixed test point:

$$\bar{f}(\mathbf{x}) := \mathbb{E}_D[\hat{f}_D(\mathbf{x})]$$

Then:

- ▶ **Bias** = $\bar{f}(\mathbf{x}) - f(\mathbf{x})$. How far off the *average* fit is from the truth. A property of the *model class*, not of any one training set.
- ▶ **Variance** = $\mathbb{E}_D\left[\left(\hat{f}_D(\mathbf{x}) - \bar{f}(\mathbf{x})\right)^2\right]$. How much the fit *wobbles* from one D to another. A property of *sensitivity to training data*.
- ▶ **Irreducible error** = σ^2 . The noise floor. No model can do better than this.

Main result: for a fixed test point $\mathbf{x}$,

$$\mathbb{E}_{D,\varepsilon}\left[(y - \hat{f}_D(\mathbf{x}))^2\right] = \text{Bias}^2[\hat{f}_D(\mathbf{x})] + \text{Var}_D[\hat{f}_D(\mathbf{x})] + \sigma^2$$

The full derivation

Plan. Add zero twice. Step 1 adds ε on one side (exposes σ^2). Step 2 adds $\bar{f}(\mathbf{x})$ (splits the rest into bias² and variance). Cross terms vanish both times.

Setup. $y = f(\mathbf{x}) + \varepsilon$, $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$, $\varepsilon \perp D$. Let $\bar{f}(\mathbf{x}) := \mathbb{E}_D[\hat{f}_D(\mathbf{x})]$. We expand $\mathbb{E}[(y - \hat{f}_D(\mathbf{x}))^2]$ (expectation over both D and ε , conditional on $\mathbf{x}$).

Step 1: separate the noise. Substitute $y = f(\mathbf{x}) + \varepsilon$:

$$\begin{aligned}\mathbb{E}[(y - \hat{f}_D(\mathbf{x}))^2] &= \mathbb{E}[(f(\mathbf{x}) - \hat{f}_D(\mathbf{x}) + \varepsilon)^2] \\ &= \mathbb{E}[(f - \hat{f}_D(\mathbf{x}))^2] + \underbrace{2 \mathbb{E}[\varepsilon (f - \hat{f}_D(\mathbf{x}))]}_{= 0 \text{ since } \varepsilon \perp D, \mathbb{E}[\varepsilon]=0} + \underbrace{\mathbb{E}[\varepsilon^2]}_{= \sigma^2}\end{aligned}$$

Step 2: add and subtract the average model $\bar{f}(\mathbf{x})$. Inside the model-error term:

$$\begin{aligned}\mathbb{E}[(f - \hat{f}_D(\mathbf{x}))^2] &= \mathbb{E}[((f - \bar{f}) + (\bar{f} - \hat{f}_D(\mathbf{x})))^2] \\ &= \underbrace{(f - \bar{f})^2}_{\text{Bias}^2, \text{ deterministic}} + 2(f - \bar{f}) \underbrace{\mathbb{E}_D[\bar{f} - \hat{f}_D(\mathbf{x})]}_{= \bar{f} - \bar{f} = 0} + \underbrace{\mathbb{E}_D[(\bar{f} - \hat{f}_D(\mathbf{x}))^2]}_{= \text{Var}_D[\hat{f}_D(\mathbf{x})]}\end{aligned}$$

Combine:

$$\mathbb{E}_{D,\varepsilon}[(y - \hat{f}_D(\mathbf{x}))^2 \mid \mathbf{x}] = \underbrace{(f(\mathbf{x}) - \bar{f}(\mathbf{x}))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_D[(\hat{f}_D(\mathbf{x}) - \bar{f}(\mathbf{x}))^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Noise}}$$

Bayes risk + the four sources of error

Bias-variance is *pointwise* (at fixed $\mathbf{x}$). A broader view, used in learning theory, decomposes the *risk* $R(f) = \mathbb{E}_{(\mathbf{x}, y)}[L(y, f(\mathbf{x}))]$ into four levels.

Let f^* = best predictor over *all* functions; f_H^* = best within your hypothesis class H ; $\hat{f}_D$ = what you actually trained.

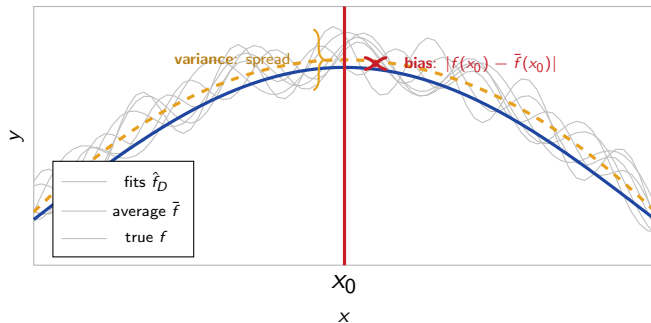
$$\underbrace{R(\hat{f}_D) - R(f^*)}_{\text{excess risk}} = \underbrace{[R(f_H^*) - R(f^*)]}_{\text{approximation}} + \underbrace{[R(\hat{f}_D) - R(f_H^*)]}_{\text{estimation+optimization}}$$

- **Bayes risk** $R(f^*) = \sigma^2$ for squared loss. The noise floor. *No model can do better*; you'd have to change the problem (collect cleaner labels) to reduce it.
- **Approximation error.** Cost of restricting to H . "A linear model cannot fit a cubic." $\approx$ bias.
- **Estimation error.** Cost of finite data D . "Same model class, but parameters differ from the population optimum." $\approx$ variance.
- **Optimization error.** Cost of incomplete convergence. "Gradient descent stopped before reaching the minimum." *Zero for OLS / Ridge / Lasso* (closed form or convex), often nonzero for NNs.

Bias-variance assumes the optimizer is perfect. For everything in L01-L05 (linear models) this holds. For NNs the optimization term reappears - and double descent (frame 18) lives in this regime.

Geometric intuition: many training sets, many fits

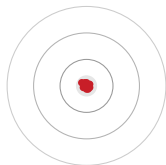
Imagine drawing 8 different training sets $D_1, \dots, D_8$ from the same distribution and fitting a model to each. The collection of fits tells you bias and variance at every x .



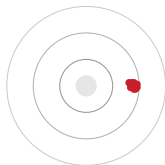
At the test point x_0 : bias is the vertical gap between the true curve and the average fit; variance is the vertical spread of the gray fits around the average.

Dartboard analogy

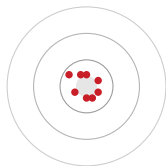
Each shot = a model fitted on a different D . The bullseye = the truth $f(\mathbf{x})$.



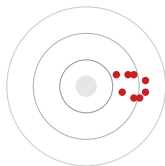
low bias, low variance



high bias, low variance



low bias, high variance



high bias, high variance

What controls bias and variance?

Lever (increase)	Bias	Var	Why
Model complexity	↓	↑	more wiggle room; fits both signal AND noise
Training data n	=	↓ ($\propto 1/n$)	average over more samples is more stable
Regularization λ	↑	↓	shrinks coefficients $\rightarrow$ less flexible, less sensitive
Features p	↓	↑	more degrees of freedom (bias if signal, variance always)
Noise σ^2	=	=	raises the irreducible floor only
Ensembling (bagging)	=	↓	averaging B models reduces variance by $\sim 1/B$

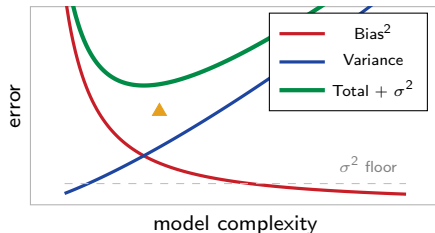
The fundamental tension: all the “classical” levers (complexity, regularization, features) trade one for the other. *Only data and ensembling reduce variance without raising bias.* Hence: “the easiest way to improve a model is to get more data.”

The classical tradeoff curve

As model complexity grows:

- ▶ Bias² **falls** (more flexibility, can match truth).
- ▶ Variance **rises** (more flexibility, fits noise too).
- ▶ Total test error follows a **U-shape**.
- ▶ The minimum is the *sweet spot*.

Cross-validation is just an empirical procedure for finding this sweet spot without knowing the curve.



Sweet spot at min of green. Dashed line: noise floor (can't go below).

Predict-first: what does more data do to bias and variance?

You have $n = 100$ samples and a fixed-complexity model. You double to $n = 200$.

Predict-first: what does more data do to bias and variance?

You have $n = 100$ samples and a fixed-complexity model. You double to $n = 200$.

Predict: what happens to

- ▶ bias?
- ▶ variance?
- ▶ irreducible noise?

Predict-first: what does more data do to bias and variance?

You have $n = 100$ samples and a fixed-complexity model. You double to $n = 200$.

Predict: what happens to

- ▶ bias?
- ▶ variance?
- ▶ irreducible noise?

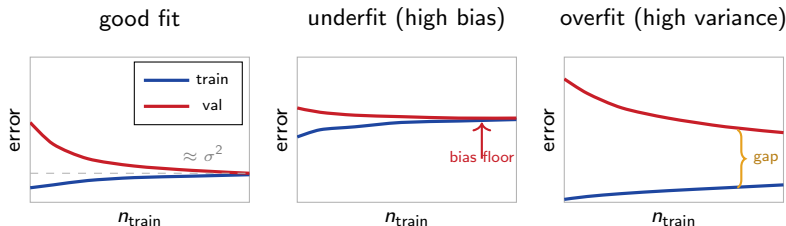
Answers:

- ▶ **Bias: unchanged.** Bias is a property of the *model class*, not the data size. A linear model still can't bend, regardless of how many points you give it.
- ▶ **Variance: roughly halves** ($\propto 1/n$ for many estimators). Each fit gets closer to the average fit when there's more data to average over.
- ▶ σ^2 : **unchanged.** The noise in $y \mid \mathbf{x}$ doesn't shrink when you collect more rows.

The operational test. Train error *low* + test error *high* $\Rightarrow$ **get more data** (it's a variance problem). Train error *also high* $\Rightarrow$ **change the model** (more data won't help; it's a bias problem). This single rule resolves most “my model isn't working” questions.

Learning curves: more data, more diagnosis

Score vs. training-set size, computed via CV. The shape tells you what's wrong.

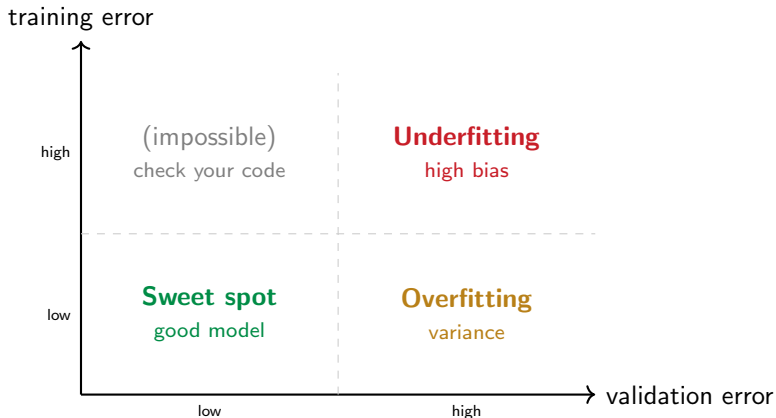


Good fit: train and val converge to a low error. Both curves close.

Underfit: train and val converge but to a *high* error. Big gap to σ^2 . More data won't help.

Overfit: train stays low, val stays high. The gap (variance) does close slowly with more data, but slowly. Add data *or* regularize.

Train vs. validation: the 4 quadrants



Concrete rules of thumb (data-dependent):

- ▶ Train $\approx$ val $\approx \sigma^2$: good. Stop.
- ▶ Train low, val high: **overfit**. More data / regularization / drop features / simpler model.
- ▶ Train and val both high: **underfit**. More complex model / features / less regularization.
- ▶ Train higher than val: bug. Probably mixed-up labels or wrong-way leakage.

Fixing underfitting (high bias)

The model class can't capture the signal. Make it more flexible. Try in order of effort:

1. **Reduce regularization first.** Smallest, fastest change. Lower λ in Ridge/Lasso (L01b, L03); raise `max_depth` on trees. If this helps, you were over-regularized.
2. **Add domain features.** Ratios, differences, interactions (L01g). Often the highest-impact change for tabular data.
3. **Polynomial expansion.** Use $\phi(\mathbf{x}) = (1, x, x^2, \dots, x^d)$ (L01b). Cheap second-order signal.
4. **Move to a more flexible model class.** Linear $\rightarrow$ tree $\rightarrow$ forest $\rightarrow$ GBM. Qualitative jump.
5. **Train longer.** For GD-style methods (NN, GBM with early stopping). If train loss is still falling, you're still learning.
6. **Remove constraints.** Larger `max_depth`, wider hidden layers, etc.

Adding data does *not* fix underfitting. Bias is a property of the model class.

Fixing overfitting (high variance)

The model is too flexible. Constrain it or drown it in data. Ranked by reliability:

1. **More data.** The single most reliable fix. Variance scales as $1/n$. If you can collect or synthesize more, do that first.
2. **Regularization.** Ridge, Lasso, ElasticNet (L01b, L03). For trees: `min_samples_leaf`, `max_depth`, `pruning`. Cheapest knob if data is fixed.
3. **Drop features.** Feature selection (L01h). Helps especially when $p > n$.
4. **Simpler model class.** Forest $\rightarrow$ shallow tree, polynomial degree 9 $\rightarrow$ 3. Qualitative reduction.
5. **Ensembling (bagging).** Average over bootstrapped models. Cuts variance without changing bias.
6. **Early stopping.** For iterative methods, halt when val score plateaus.
7. **CV-tune the hyperparameters** you control with all of the above (L01d, L01e).

Regularization as the bias-variance lever

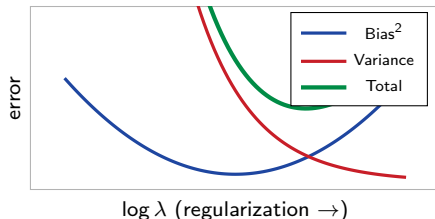
Regularization λ is the cleanest knob. With Ridge regression:

$$\hat{\theta}_{\text{Ridge}} = \arg \min_{\theta} \|\mathbf{y} - \mathbf{X}\theta\|^2 + \lambda \|\theta\|^2$$

- ▶ $\lambda = 0$: pure OLS. Lowest bias, highest variance.
- ▶ $\lambda \rightarrow \infty$: $\hat{\theta} \rightarrow 0$. Highest bias (predicts $\bar{y}$), zero variance.
- ▶ Intermediate λ : trades bias against variance smoothly.

Pick λ at the minimum of CV error.

Same U as complexity, x-axis flipped: λ is the *inverse* of model capacity. Two knobs, one tradeoff.



Cross-validation as a bias-variance manager

CV is itself subject to bias-variance considerations. Choosing k trades them:

- ▶ $k = n$ (**LOO**). Train sets are nearly identical $\rightarrow$ correlated fold scores $\rightarrow$ high *variance* of the CV estimate. Bias of the CV estimate is low (uses $n - 1$ samples per fit, almost the whole training set).
- ▶ $k = 5$ **or** 10. Train sets share less data $\rightarrow$ less correlated $\rightarrow$ lower variance. Each fit uses 80%-90% of the data $\rightarrow$ small bias.
- ▶ $k = 2$. Each fit uses only half the data $\rightarrow$ *biased* (CV scores look worse than the model trained on full data) but low variance.

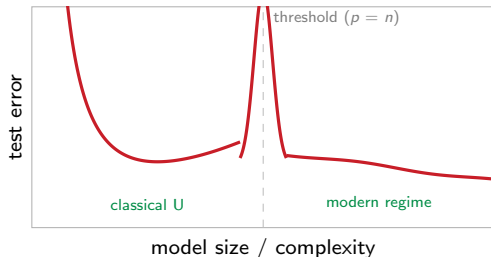
Standard $k = 5$ is the sweet spot for the CV estimate's own bias-variance tradeoff (the same U-shape, at a different level).

Takeaway: you use CV to find the bias-variance sweet spot *for the model*; meanwhile your choice of k trades bias and variance *for the CV estimate itself*. Bias-variance is fractal.

Double descent

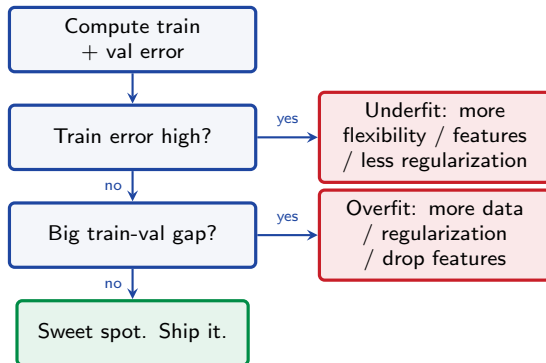
Classical theory: test error has a single U-shape vs. complexity. *Modern* models (deep nets, very-wide-feature linear models) show a **second descent**.

- ▶ Up to the **interpolation threshold** ($p \approx n$): test error follows the classical U.
- ▶ At $p \approx n$: test error *peaks*. The model can just barely fit the training data, and does so with huge coefficients.
- ▶ Beyond ($p \gg n$): test error *falls again*. There are many ways to fit the training data exactly; gradient descent happens to prefer low-norm solutions, and those generalize. (The technical name is “implicit regularization”.)



Why students care: this is why deep nets work even though they're massively over-parameterized. Classical bias-variance is the small- p half of a bigger picture.

Diagnosis workflow



Iterate: apply a fix, retrain, recheck the gap. Stop when CV error plateaus.

Recap

- ▶ **Two failure modes:** underfit (high bias) vs. overfit (high variance).
- ▶ **The decomposition:** $\mathbb{E}[(y - \hat{f}_D(\mathbf{x}))^2] = \text{Bias}^2 + \text{Var} + \sigma^2$, derived in 4 lines via two “add-and-subtract” tricks.
- ▶ **Levers:** complexity, regularization, features all trade bias for variance. Only *data* and *ensembling* reduce variance without raising bias.
- ▶ **Diagnosis:** train vs. val error gap. Both high = underfit; gap big = overfit.
- ▶ **CV** finds the bias-variance sweet spot empirically. The choice of k trades bias and variance *of the CV estimate itself* - typically $k = 5$.
- ▶ **Modern wrinkle:** beyond the interpolation threshold, test error descends again (deep nets). Classical bias-variance is the small- n half of the picture.
- ▶ **When to stop.** Train-val gap is small AND CV error has plateaued AND the operational test (frame 11) says “no easy fix left”. Then ship.

HW01d2 - empirical bias-variance

Estimated time: 90-120 minutes. Tasks 1-4 graded; bonuses optional.

1. Generate $n = 30$ noisy samples from $y = \sin(\pi x) + \mathcal{N}(0, 0.2^2)$ on $x \in [0, 1]$. Fit polynomial regression for degrees $d = 1, 2, \dots, 15$. Plot training MSE and test MSE vs. d . Identify the sweet spot.
2. **Empirical bias-variance.** Repeat $M = 500$ times: regenerate training data, refit each degree. At a held-out test grid, compute mean prediction $\bar{f}_M$ and variance Var_M . Estimate empirical Bias^2 using $f(x) = \sin(\pi x)$. Plot $\widehat{\text{Bias}}^2$, $\widehat{\text{Var}}$, $\widehat{\text{Bias}}^2 + \widehat{\text{Var}} + \sigma^2$ vs. d . Verify the third curve overlaps your test MSE from Task 1.
3. Vary $n \in \{10, 30, 100, 300\}$. Re-run Task 2 for each. Plot bias and variance vs. n . Confirm: bias roughly constant, variance $\propto 1/n$.
4. Fit Ridge with degree-15 features and $\lambda \in \{10^{-4}, 10^{-2}, 1, 10^2, 10^4\}$. Empirically decompose bias-variance for each λ . Where is the Ridge sweet spot?
5. *Bonus 1:* Simulate double descent. Use a degree-50 polynomial on $n = 30$ samples with no regularization. Plot test error as you add features one at a time. Look for the spike near $p = n$.
6. *Bonus 2:* Pair Task 2 with bagging: average predictions from 50 Ridge models trained on bootstrap samples. Show variance drops, bias roughly constant.
7. *Bonus 3:* Repeat Task 2 with a tree of varying `max_depth`. Compare the bias-variance